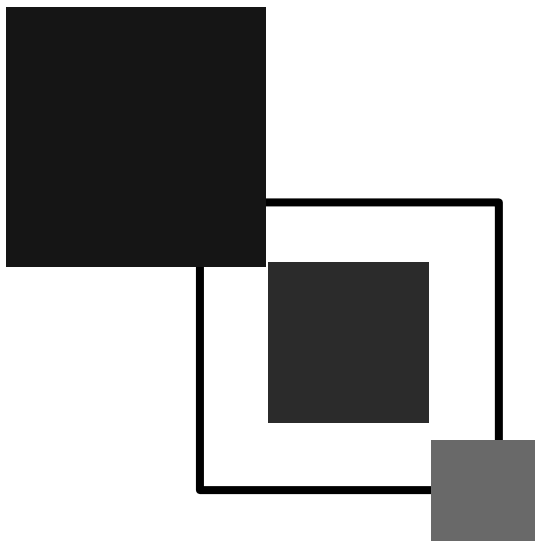
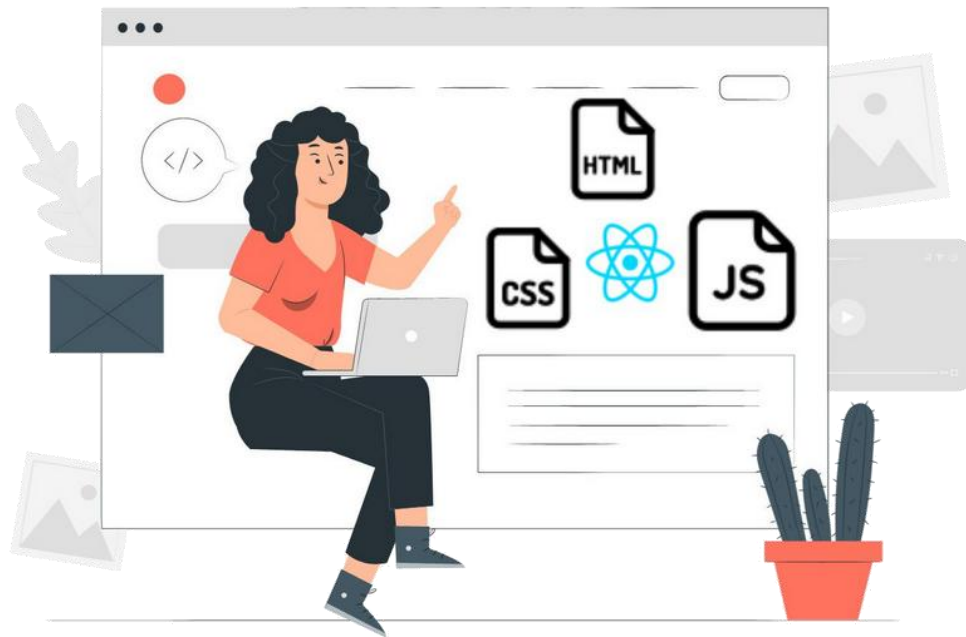




Banco de Dados Marion e Isack

Etec Prof. Horácio Augusto da Silveira

2024



FERRAMENTAS DE TRABALHO



MongoDB

Banco de dados NoSQL
Conceitos



mongoDB®

MONGO DB

Ambiente de desenvolvimento integrado
(IDE) usado para manipular dados



01

BANCO DE DADOS NoSQL

Conceitos básicos

OPERADORES LÓGICOS

Podemos utilizar operadores lógicos nas buscas, sendo eles: \$or, \$and, \$nor, \$ne, \$not

- **\$or** retorna documentos pesquisados quando pelo menos uma das cláusulas de comparação for verdadeira.
- **\$and** retorna documentos pesquisados quando todas as cláusulas de comparação for verdadeira.
- **\$nor** retorna documentos pesquisados que não satisfaçam as cláusulas de comparação.
- **\$ne** retorna documentos pesquisados cujos valores sejam diferentes do valor informado.
- **\$not** efetua uma operação not.

EXIBINDO COLEÇÕES

Comando em SQL	Comando em MongoDB
SELECT * FROM emp WHERE idade >21 or salario < 12500 ;	db.emp.find({\$or:[{Idade:{\$gt:21}},{Salario:{\$lt:12500}}]});
SELECT * FROM emp WHERE idade > 21 and salario < 12500	db.emp.find({\$and:[{Idade:{\$gt:21}},{Salario:{\$lt:12500}}]}); db.emp.find({Idade: {\$gt: 21},Salario:{\$lt: 12500}});
SELECT * FROM emp WHERE NOT (Idade > 22 OR Salario > 12500);	db.emp.find({\$nor:[{Idade:{\$gt:22}},{Salario:{\$gt:12500}}]});
SELECT * FROM emp WHERE idade <> 22	db.emp.find({Idade:{\$ne:22}});
SELECT * FROM emp WHERE NOT (Idade > 21);	db.emp.find({Idade:{\$not:{\$gt:21}}});

OPERADORES PARA TRATAMENTO ARRAYS

Podemos utilizar operadores para tratar arrays, são eles: \$in, \$nin, \$all, \$exists

- **\$in** retorna documentos nos quais o valor pesquisado esteja na lista de valores.
- **\$nin** retorna documentos nos quais o valor pesquisado não esteja na lista de valores ou o campo pesquisado não exista no documento.
- **\$all** retorna documentos nos quais todos valores pesquisados estejam na lista de valores.
- **\$exists** não específico para o uso em arrays, mas pode ser usado para complementar os operadores \$in e \$nin. É usado para testar se um campo existe em um documento ou não.

EXIBINDO COLEÇÕES

Comando em SQL	Comando em MongoDB
SELECT * FROM emp WHERE dependentes IN ('Rosa', 'Hugo');	db.emp.find({Dependentes:{\$in:["Rosa", "Hugo"]}});
SELECT * FROM emp WHERE dependentes NOT IN ('Rosa', 'Rita');	db.emp.find({Dependentes:{\$nin:["Rosa", "Rita"]}});
SELECT * FROM emp WHERE dependents LIKE '%Rita%' AND Dependentes LIKE '%Ana%';	db.emp.find({Dependentes:{\$all:["Rita", "Ana"]}});
SELECT * FROM emp WHERE Dependentes IS NOT NULL;	db.emp.find({Dependentes:{\$exists: true}}).count() db.emp.find({Dependentes:{\$exists: false}}).count()

No exemplo, usamos o método `count()` que efetua a contagem do número de elementos retornados

EXPRESSÕES REGULARES

Em um banco MongoDB, expressões regulares permitem pesquisar por padrões em campos texto. Há dois tipos de sintaxe de expressões regulares.

```
{ <campo>: { $regex: /padrão/, $options: '<opções>' } }  
{ <campo>: { $regex: /padrão/, $options: '<opções>' } }  
{ <campo>: { $regex: /padrão/ <opções> } }
```

OU

```
{ <campo>: padrão/ <opções> }
```


EXPRESSÕES REGULARES

Em que:

- **Campo:** indica o campo no qual será pesquisado o padrão.
- **Padrão:** indica o padrão a ser pesquisado.
- **Opções:** indica as opções a serem usadas com a expressão regular. Algumas das opções são:

i: indica case insensitive, isto é, o texto pode estar tanto em letras maiúsculas quanto em letras minúsculas.

m: usado em padrões que incluem âncoras, isto é, ^ para o início de um texto e \$ para o final do texto.

x: usado para ignorar todos os espaços em branco no padrão.

EXPRESSÕES REGULARES

Significado	Comando em MongoDB
Pesquisar todos nomes iniciados pela letra B maiúscula.	<code>db.emp.find({Nome:{\$regex:"^B"}});</code>
Pesquisar todos nomes iniciados pela letra B, independente do caps lock.	<code>db.emp.find({Nome:{\$regex:"^B", \$options: 'i'}});</code>
Pesquisar todos os nomes que possuem a letra “i”, minúscula, em qualquer parte do nome.	<code>db.emp.find({Nome:{\$regex:"i"}});</code>
Pesquisar todos os nomes que possuem a letra “i”, independente do caps lock, em qualquer parte do nome.	<code>db.emp.find({Nome:{\$regex:"i", \$options: 'i'}});</code>
Pesquisar todos os nomes terminados pela letra “s”, minúscula.	<code>db.emp.find({Nome:{\$regex:"s\$"}});</code>

ALTERANDO DADOS

O método `update()` permite alterar um ou mais documentos em uma coleção. É possível alterar um ou mais campos de um documento ou, até mesmo, o documento inteiro. Por padrão altera um documento por vez.

```
db.coleção.update(consulta, atualização, opções)
```

Em que:

- **Coleção:** indica o nome da coleção.
- **Consulta:** indica quais serão os critérios para pesquisar o documento que será alterado.
- **Atualização:** indica a modificação que será feita.

ALTERANDO DADOS

- **Opções:** indica quais são as opções para o comando. Entre as opções estão:
 - **Upsert:** opcional, quando definido como true, cria um novo documento, caso nenhum documento atenda os critérios de busca, o padrão é false.
 - **Multi:** opcional, se definido como true, atualiza todos os documentos que atendam os critérios de busca, padrão é false.

ALTERANDO DADOS

O método update tem operadores para tratar os campos individualmente, sendo eles:

- **\$inc**: incrementa o valor de um campo pelo valor informado no operador.
- **\$set**: modifica o conteúdo de um campo específico. Caso o campo não exista, um novo campo será acrescentado ao documento.
- **\$rename**: renomeia um campo.
- **\$unset**: remove um campo específico de um documento.

EXPRESSÕES REGULARES

Exemplo em SQL	Comando em MongoDB
UPDATE emp SET Idade = Idade + 2 WHERE Nome = 'Ana'	db.emp.update ({Nome: "Ana"}, {\$inc: {Idade: 2}}, {multi: true})
UPDATE emp SET Idade = 25 WHERE Nome = 'Ana'	db.emp.update ({Nome: "Ana"}, {\$set: {Idade: 25}}, {multi: true})
Outros exemplos	Comando em MongoDB
Remove o campo salário do documento	db.emp.update({Nome:"Ana"},{\$unset:{salario:""}});
UPDATE emp SET Idade = 25 WHERE Nome = 'Ana'	db.emp.update ({Nome: "Ana"}, {\$set: {Idade: 25}}, {multi: true})

EXCLUINDO DADOS

O método `deleteOne()` exclui todos os documentos que atendam as condições de pesquisa especificadas. A opção `justOne` limita a operação de remoção a um único documento. Sua sintaxe é:

```
db.coleção.deleteOne(consulta, { justOne:<boolean> } )
```

Em que:

- **Coleção**: indica o nome da coleção.
- **Consulta**: indica quais serão os critérios para pesquisar o documento que será removido.
- **justOne**: indica que apenas um documento será removido.

Caso queira excluir a coleção use o método `drop()` .

EXCLUINDO DADOS

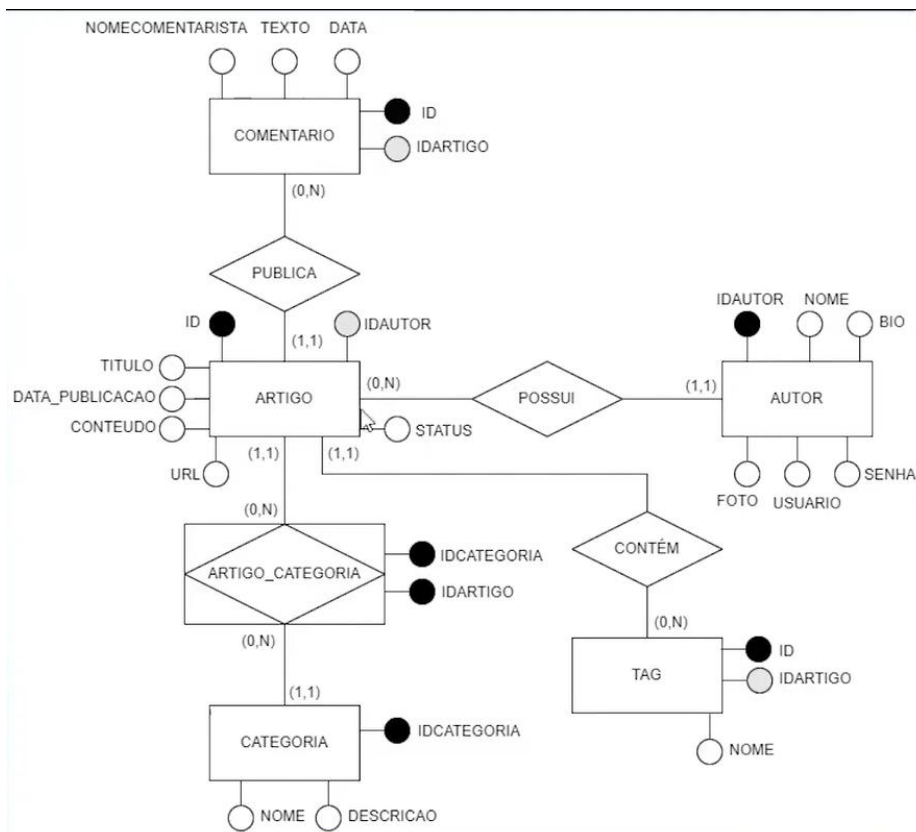
```
db.apaga.insertOne({cod:7369,nome:"Maria",cargo:"DBA"});  
db.apaga.insertOne({cod:7499,nome:"Rosa",cargo:"DBA"});  
db.apaga.insertOne({cod:7521,nome:"Ana",cargo:"DBA"});  
db.apaga.insertOne({cod:7566,nome:"Rita",cargo:"DBA"});  
db.apaga.find();
```

```
db.apaga.deleteOne({cod:{ $gt:7521}});  
db.apaga.find();
```

```
db.apaga.deleteOne({carg:"DBA"},true);  
db.apaga.find();
```

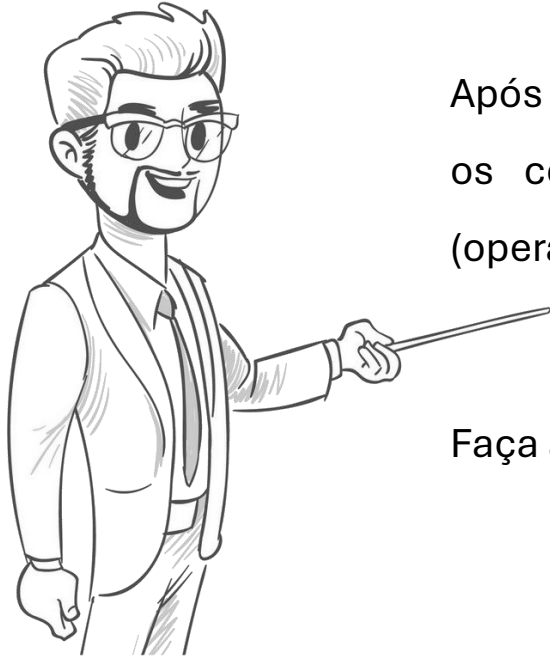
```
db.apaga.deleteOne({});  
db.apaga.find();
```


Vamos praticar



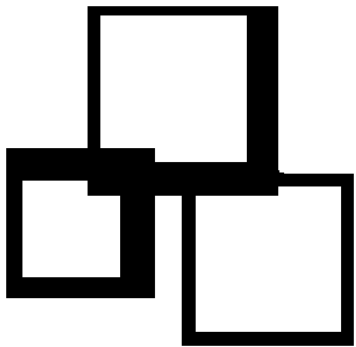
Vamos praticar!

Considerando a **Base de Dados: Blog**



Após a criação dos documentos elabore 10 consultas usando os conceitos abordados na aula, um exemplo de cada (operadores lógicos: não, e, ou), expressões regulares.

Faça alterações e exclusões.



OBRIGADO

To be continued...

BYE
BYE